Access-Control-Allow-Credentials, which indicates that cookies may be included with requests, and Access-Control-Expose-Headers, which indicates which response headers are visible to the client.

We'll start by having you add a login page to the sample blogging app. The login page will include a cookie that needs to be validated in order to delete blog posts. Next, you'll modify the sample to display a response header on the page. By setting the Access-Control-Allow-Credentials and Access-Control-Expose-Headers headers on the response, the sample app will enable support for each of these features.

5.1 Supporting cookies in CORS requests

The sample app has a pretty glaring security hole: anyone visiting the site can delete a blog post! The server code doesn't verify whether the user deleting the post has permission to do so. Regardless of whether you're the blog author or a casual reader, clicking the Delete link will delete a post. This is obviously something that needs to be fixed.

The app needs a way to identify the user, and the standard way of identifying a user on the web is to use cookies. Cookies are small bits of information exchanged between the server and the browser. For example, when a user logs in to Facebook, the Facebook server sets a cookie with a unique user ID. As the user browses the site, each request to the server contains the cookie. The server can read the cookie from the request and load user-specific data, such as the user's timeline.

> **NOTE** The CORS spec uses the term *user credentials* to describe any bits of information the browser sets on a request to identify the user. This includes cookies, basic HTTP authentication, and client-side SSL. While the techniques described in this section apply to all of these types of user credentials, the text will focus only on cookies to keep things simple.

As you can see from this example, using cookies revolves around two actions: setting the cookie and reading it. A login page can be used to validate the user and set the appropriate cookie. Once the cookie is set, any user-specific activity should first read the cookie and validate the user. Let's modify the sample app to support these two actions. We'll start with the login page.

5.1.1 Setting cookies with a login page

The sample app currently only has one page, client.html, where users can view blog posts. In this section you'll add a second page called login.html. This login page will collect the user's username and password and set a cookie on his or her behalf.

> **NOTE** This code is only meant as an example. It's not a best practice for implementing security for your own site.

Figure 5.1 shows how the login page integrates with the rest of the app. The login page lives on 127.0.0.1:9999, along with the API. Because the login page lives on 127.0.0.1:9999, the cookie will be set on that domain as well.